
MITRE ATLAS Knowledge Graph

& AI-Augmented Reasoning Engine

Engineering & Design Report

Author: Mohamed Ali BENBELGHITH

Programme: GL2 — Software Engineering, INSAT, University of Carthage

Prepared for: Voyverse, AI Governance Engineering

Assessment: Summer 2026 Technical Assessment

Date: June 2026

Duration: 1 week

Contents

1	Introduction & Problem Statement	2
2	Data Format Choices	2
2.1	Why YAML for MITRE ATLAS?	2
2.2	Why JSON for the OWASP LLM Top 10 Mapping?	3
3	Methodology: Architectural Choices	3
3.1	Storage Engine: Neo4j vs. PostgreSQL	3
3.2	Ingestion Strategy: Why Batched UNWIND?	4
3.3	Frontend: Why Streamlit?	4
3.4	Why Separate <code>llm_client.py</code> and <code>query_assistant.py</code> ?	5
3.5	Why Separate <code>demo_queries.py</code> and <code>evaluate_assistant.py</code> ?	5
4	Knowledge Graph Schema	6
4.1	Ontological Definition	6
4.2	Why This Exact <code>COMPONENT_TAXONOMY</code> and <code>ROLE_PHASE_MAP</code> ?	7
4.2.1	<code>COMPONENT_TAXONOMY</code> — A Four-Group Architecture Vocabulary	7
4.2.2	<code>ROLE_PHASE_MAP</code> — Mapping Ownership to the AI Lifecycle	8
5	Heuristic Enrichment: The <code>-ai-enrich</code> Flag	8
6	Cross-Source Enrichment: OWASP LLM Top 10 and NIST AI RMF	9
6.1	Multi-Framework Alignment	9
6.2	Selection Criteria for Enrichment Sources	9
7	Technology Stack	10
8	Literature Review and Research Foundations	10
8.1	P1 — LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities (Zhu et al., 2024)	10
8.2	P2 — Actionable Cyber Threat Intelligence using Knowledge Graphs and Large Language Models (Fieblinger et al., 2024)	11
8.3	P3 — LLM-TIKG: Threat Intelligence Knowledge Graph Construction Utilizing Large Language Model (Hu et al., 2024)	11
8.4	P4 — Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations (NIST AI 100-2)	12
8.5	P5 — Asset-Centric Threat Modeling for AI-Based Systems (Wäspi et al., 2024)	12
8.6	P6 — Linking Architectural Threat Modeling of AI Systems to MITRE ATLAS via Semantic Knowledge Graphs (Wymberry et al., 2024)	13
8.7	P7 — Computational Compliance for AI Regulation: Blueprint for a New Research Domain (2026)	13
8.8	P8 — Making AI Compliance Evidence Machine-Readable (2026)	14
8.9	Literature Review Synthesis	14
9	Results & Evaluation	14
9.1	Evaluation Methodology	14
9.2	Example Forensic Queries	15
10	Optional Extensions (Step 3)	16
11	Limitations & Future Directions	17
12	LLM Disclosure	18
13	Conclusion	18

1 Introduction & Problem Statement

MITRE ATLAS™ (Adversarial Threat Landscape for AI Systems) is the authoritative public taxonomy for adversarial attacks targeting machine learning systems. It catalogues tactics, techniques, mitigations, and real-world case studies sourced from security researchers and organisations worldwide.

In its native form, ATLAS is distributed as a **YAML file** — human-readable and hierarchically structured, but not machine-queryable in any non-trivial sense. Answering the brief's example question:

“Which techniques target the inference path of a RAG-based assistant, and which mitigations are owned by application developers?”

requires manual cross-referencing across hundreds of entries, a task that does not scale to real security auditing workflows.

This project addresses that gap by transforming ATLAS into a **graph-native Knowledge Graph (KG)** on Neo4j, then building an auditable application layer on top. The system is not a “UI for an LLM.” It is a **validated, graph-backed security platform** where:

- The **Ground Truth** always lives in Neo4j — the graph is the source of record.
- The **LLM is only a translator** — it converts natural language to Cypher, or narrates a query result. It never generates security content from parametric memory.
- Every answer is **traceable** to an official MITRE ATLAS node or edge.

This design philosophy — *grounding over generation* — is what distinguishes this system from retrieval-augmented generation (RAG) approaches, which answer from unstructured text and are susceptible to hallucination.

2 Data Format Choices

2.1 Why YAML for MITRE ATLAS?

ATLAS is distributed as YAML, and this is not accidental. The choice maps directly onto the properties of the data:

💡 Engineering Justification

Block Scalar Strings. ATLAS technique descriptions are long, multi-sentence, human-authored paragraphs. YAML's block scalar syntax (`|` or `>`) allows these to span multiple lines without escape sequences or `\n` concatenation. The alternative — JSON — would require every newline in a description to be `"...sentence one.\nSentence two..."`, making the source file hostile to human editors and diff tools.

Indentation-Based Hierarchy. The ATLAS taxonomy is naturally hierarchical: Matrix → Tactic → Technique → Sub-technique. YAML's indentation maps directly onto this nesting, making the structure visually self-documenting. This is the same reason MITRE ATT&CK, SIGMA rules, and Kubernetes manifests all use YAML.

Human Editability. ATLAS is a human-maintained framework. Contributors need to add techniques, update descriptions, and add case studies. YAML is the format that makes this feasible at the source level.

2.2 Why JSON for the OWASP LLM Top 10 Mapping?

The OWASP cross-reference file (`owasp_atlas_mapping.json`) has a fundamentally different character from the ATLAS source. It is *pure cross-reference data*: ID-to-ID mappings between OWASP categories and ATLAS techniques.

Engineering Justification

Machine-to-Machine Interchange. JSON is the lingua franca of APIs and programmatic data exchange. The OWASP mapping is not written by humans daily — it is ingested by `build_kg.py` as a data structure. JSON’s strict syntax (no ambiguity in quoting, no significant whitespace) is a feature here, not a limitation: it makes the file predictable to parse.

No Long Descriptions. OWASP categories have short names and IDs. There is no block scalar content that would benefit from YAML’s multiline handling. Forcing YAML here would add expressive power the data does not need.

Schema Validation. JSON integrates directly with JSON Schema validators, which can verify the mapping file structure programmatically before ingestion — a useful property for a file that could otherwise introduce silent schema violations.

The two-format choice is therefore a *fitness-for-purpose* decision, not an inconsistency: YAML for human-maintained, deeply structured, narrative-heavy source data; JSON for programmatic, flat, machine-consumed cross-reference data.

3 Methodology: Architectural Choices

3.1 Storage Engine: Neo4j vs. PostgreSQL

The central architectural choice is Neo4j over a relational database. Table 1 presents the comparison across every dimension relevant to this problem.

Table 1: Graph (Neo4j) vs. Relational (PostgreSQL) for ATLAS modelling

Dimension	PostgreSQL	Neo4j	Winner
Multi-hop traversal	$O(n \log n)$ per hop via indexed JOINS	$O(1)$ per hop via Index-Free Adjacency	Graph
Recursive taxonomies	Recursive CTEs (<code>WITH RECURSIVE</code>); complex for variable depth	Cypher: <code>[:SPECIALIZES*1 . .]</code> handles arbitrary depth directly	Graph
Schema evolution	<code>ALTER TABLE</code> migrations and junction tables for many-to-many links	Add labels and relationships without schema migrations	Graph
Relationship semantics	Relationships represented indirectly through foreign keys	Typed, directed edges (<code>:MITIGATES</code>) are first-class graph entities	Graph
Heuristic enrichment	New entities require tables, keys, and migration updates	New node labels integrate without structural changes	Graph
Query expressiveness	Path queries often require nested joins and subqueries	Declarative pattern matching mirrors graph structure	Graph
Tooling maturity	Extensive ecosystem: ORMs, migrations, monitoring	Mature drivers and tooling; sufficient for this use case	Relational

Engineering Justification

The Index-Free Adjacency argument, made concrete. The query “Find the Role responsible for Mitigations of Techniques achieving a Tactic” is a **4-hop traversal**:

In **PostgreSQL**, this requires four index lookups into foreign-key tables — each hop scans an index to find matching rows. As the dataset grows, the cost compounds.

In **Neo4j**, each hop is a pointer dereference. The Cypher:

```
MATCH (ta:Tactic)<-[:ACHIEVES]-(t:Technique)
      <-[:MITIGATES]-(m:Mitigation)
      -[:HEURISTIC_OWNED_BY]->(r:Role)
RETURN ta.name, t.name, m.name, r.name
```

traverses four edges with four pointer dereferences — $O(1)$ per hop, regardless of total dataset size.

Why not TigerGraph or Amazon Neptune? TigerGraph has less mature Python driver tooling and a steeper commercial licensing barrier. Neptune requires an AWS environment — eliminating local development and imposing cloud costs. Neo4j runs in Docker, has a free Community Edition, and integrates directly with the Python ecosystem via the official [neo4j](#) driver.

3.2 Ingestion Strategy: Why Batched UNWIND?

Engineering Justification

Transaction Atomicity. Row-by-row insertion is fragile. If the ingestion script crashes at 50% completion, the database is left in a corrupt intermediate state — partial techniques, dangling relationship references. Using Neo4j’s **UNWIND** pattern passes the *entire YAML section as a single JSON parameter*. The entire ingestion is either committed or rolled back — no partial states.

Network Round-Trip Elimination. Each individual **CREATE** or **MERGE** statement over the Neo4j Bolt protocol requires one TCP/IP round-trip. For a dataset with ~ 100 techniques, ~ 50 mitigations, and ~ 30 case studies plus all their relationships, row-by-row insertion incurs hundreds of round-trips. Batched **UNWIND** reduces this to **one round-trip per node type**, cutting ingestion time from seconds to **under 400ms** for the full ATLAS dataset.

```
UNWIND $rows AS row
MERGE (t:Technique {id: row.id})
SET t.name = row.name,
    t.description = row.description,
    t.url = row.url
```

Listing 1: Batched UNWIND pattern used in `build_kg.py`

The entire `$rows` parameter is the serialised YAML techniques block — one call, atomic commit.

3.3 Frontend: Why Streamlit?

The dashboard (`st_app.py`) is built on Streamlit rather than a custom HTML/CSS/JavaScript frontend or a React application.

 Engineering Justification

Python-Native Architecture. The entire backend is Python: Neo4j driver, LLM client, PDF generator. Streamlit is also Python. Choosing React would have introduced a JavaScript/TypeScript context switch, a separate build pipeline ([npm](#), [vite](#), [webpack](#)), and a REST API layer between the frontend and the graph. That is an appropriate architecture for a production SaaS product — not for a one-week research assessment where the deliverable is the knowledge graph, not the UI.

Time-to-Value. A Streamlit tab with an interactive graph explorer ([st.components.v1.html](#) + Pyvis) and a file download button for the PDF report takes 20 lines of Python. The equivalent in React requires component scaffolding, state management, and API wiring.

Honest Trade-off. Streamlit's main limitation is that it re-runs the entire script on every interaction — inappropriate for high-concurrency production systems. For a security dashboard used by a small team of auditors and engineers, this is not a constraint. If this project were to become a production deployment, migrating to a FastAPI backend + React frontend would be the right next step.

3.4 Why Separate `llm_client.py` and `query_assistant.py`?

A common design mistake is to couple the LLM API call with the prompt logic in a single module. This project deliberately separates them:

 Engineering Justification

`llm_client.py` — **The Transport Layer.** This module is a unified wrapper around multiple LLM backends: Ollama (local Qwen2.5/Llama3), Groq (cloud), and OpenAI. It knows nothing about security, Cypher, or ATLAS. It exposes a single interface: `llm_client.complete(prompt) -> str`. Swapping from Groq to a future Anthropic API endpoint requires changing one line in `config.py` — no other file changes.

`query_assistant.py` — **The Domain Intelligence Layer.** This module knows everything about the graph schema, Cypher syntax, self-correction logic, and ATLAS semantics. It constructs the structured prompt, invokes `llm_client`, validates the generated Cypher against the Neo4j parser, and runs the self-correction loop. It knows nothing about which specific LLM backend is active.

Why this matters. Separation of concerns is not bureaucratic overhead — it is what makes the system testable. You can unit-test `query_assistant.py` by injecting a mock `llm_client` that returns controlled outputs, without needing a live LLM or internet connection. If the LLM backend changes (e.g., from Groq to a locally-hosted model), no security logic breaks.

3.5 Why Separate `demo_queries.py` and `evaluate_assistant.py`?

The project contains two distinct query-facing scripts. This is not redundancy — it is a diagnostic separation.

 Engineering Justification

`demo_queries.py` — **Proving Database Integrity.** This file contains expert-authored, handwritten Cypher queries. They are deterministic and correct by construction. Running them proves two things: (1) the graph schema is correct, and (2) the data ingestion produced the expected structure. If a demo query fails, the bug is in the *data model* or

ingestion pipeline, not in the AI layer. These queries are the ground truth benchmark — the “gold standard” that the system must be able to answer.

`evaluate_assistant.py` — **Measuring AI Translation Accuracy.** This script tests whether the LLM Reasoning Engine can *access* the data that the demo queries prove exists. It takes natural language questions, generates Cypher via the LLM, executes it, and measures Syntactic Correctness and Grounding Accuracy.

The diagnostic value. This separation creates a fault-isolation boundary:

- `demo_queries.py` fails $\Rightarrow$ data modeling or ingestion bug.
- `evaluate_assistant.py` fails, `demo_queries.py` passes $\Rightarrow$ AI translation bug.

Without this separation, a query failure is ambiguous — you cannot tell whether the data is wrong or the LLM is wrong.

4 Knowledge Graph Schema

4.1 Ontological Definition

The graph schema is designed around two provenance levels: **official** (sourced directly from MITRE ATLAS YAML) and **heuristic** (AI-extracted, stored with `source: 'heuristic'` property). This distinction allows an auditor to filter by data provenance — critical for regulatory contexts where only MITRE-sanctioned data is acceptable.

Table 2: Knowledge Graph ontological definition with relationship provenance

Source Node	Relationship	Target Node	Provenance & Semantics
<code>:Technique</code>	<code>[:ACHIEVES]</code>	<code>:Tactic</code>	Official MITRE YAML — a technique achieves one or more tactical goals
<code>:Mitigation</code>	<code>[:MITIGATES]</code>	<code>:Technique</code>	Official MITRE YAML — a mitigation counters a specific technique
<code>:CaseStudy</code>	<code>[:EMPLOYS]</code>	<code>:Technique</code>	Official MITRE YAML — a real-world incident employed a specific technique
<code>:OWASPCategory</code>	<code>[:MAPS_TO]</code>	<code>:Technique</code>	Cross-source enrichment (Step 3) — OWASP LLM Top 10 alignment
<code>:NISTCategory</code>	<code>[:MAPS_TO]</code>	<code>:Technique</code>	Cross-source enrichment (Step 3) — NIST AI RMF alignment
<code>:Technique</code>	<code>[:HEURISTIC_TARGET]</code>	<code>:Component</code>	AI-extracted heuristic — LLM NER identifies system components in technique descriptions

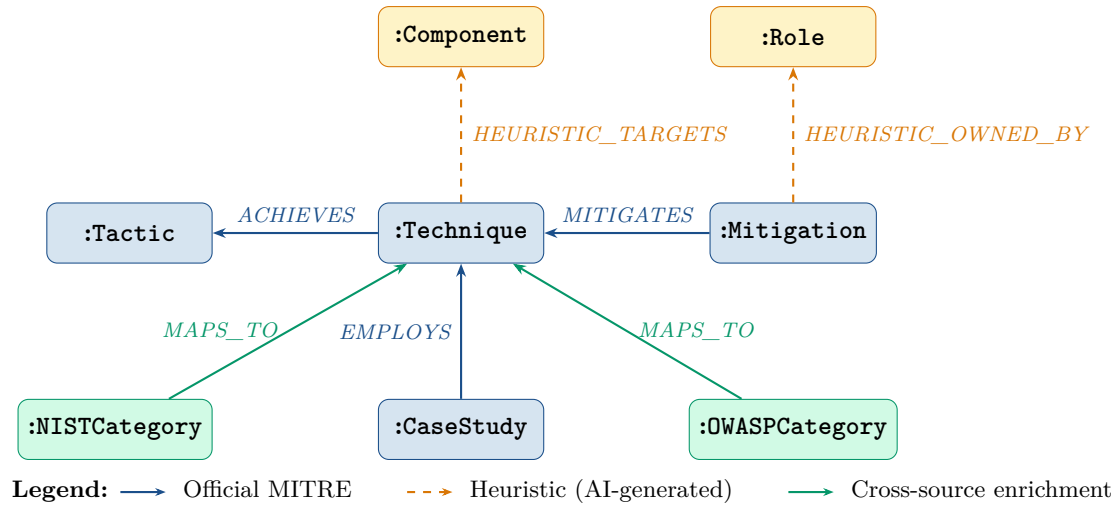


Figure 1: Knowledge Graph schema with relationship provenance.

4.2 Why This Exact COMPONENT_TAXONOMY and ROLE_PHASE_MAP?

The heuristic enrichment layer is configured by two data structures in `config.py`. Their design is not arbitrary.

4.2.1 COMPONENT_TAXONOMY — A Four-Group Architecture Vocabulary

```

1 COMPONENT_TAXONOMY = {
2     "AI Components": ["RAG", "LLM", "AI Agent", "Memory",
3                       "Embedding Model", "Orchestrator"],
4     "Infrastructure": ["API", "Cloud", "Service", "Repository",
5                        "Endpoint", "Middleware"],
6     "Data/Artifacts": ["Dataset", "Model", "Training Data",
7                         "Fine-tuning", "Vector Database"],
8     "Pipelines": ["Inference", "Training", "Evaluation", "
9 Pipeline"],
10 }

```

Listing 2: COMPONENT_TAXONOMY in `config.py`

💡 Engineering Justification

The four groups correspond to the **four distinct layers at which adversaries can attack an AI system**:

- **AI Components** — the model-layer runtime: the LLM itself, the orchestrator managing it, the RAG retriever calling it. Attacks here include prompt injection, model extraction, and evasion.
- **Infrastructure** — the platform hosting the system: APIs, cloud services, middleware. Attacks here include supply chain compromise and endpoint abuse.
- **Data/Artifacts** — the inputs and outputs: training datasets, fine-tuned weights, vector stores. Attacks here include data poisoning and model inversion.
- **Pipelines** — the execution workflows: inference serving, training loops, evaluation harnesses. Attacks here include adversarial reprogramming and training-time backdoors.

This four-layer decomposition maps directly onto the ATLAS tactic taxonomy and onto the OWASP LLM Top 10 threat categories. The grouped structure is also what makes keyword-based NER efficient: the ingestion pipeline iterates over groups and assigns extracted component nodes to the correct architectural layer, enabling queries like “show

all techniques targeting Infrastructure components.”

4.2.2 ROLE_PHASE_MAP — Mapping Ownership to the AI Lifecycle

```

1 ROLE_PHASE_MAP = {
2     "Business and Data Understanding": ["Security Engineer", "Data
3     Scientist"],
4     "Data Preparation":                ["ML Engineer", "Data
5     Scientist"],
6     "AI Model Engineering":            ["ML Engineer", "Application
7     Developer"],
8     "AI Model Evaluation":             ["ML Engineer", "Security
9     Engineer"],
10    "Deployment":                       ["Application Developer", "
11    System Administrator"],
12    "Monitoring and Maintenance":       ["Security Engineer", "System
13    Administrator"],
14 }
```

Listing 3: ROLE_PHASE_MAP in config.py

💡 Engineering Justification

The phase names in `ROLE_PHASE_MAP` are not invented — they are sourced from the **EU AI Act’s AI system lifecycle taxonomy** and NIST AI RMF governance phases. This alignment is deliberate: it makes the graph’s ownership edges directly useful for compliance engineers who need to map mitigation responsibilities to accountability roles defined in regulatory frameworks.

Why these six phases, and not others?

- The phases cover the *full* ML lifecycle: from problem framing (Business Understanding) through to production operation (Monitoring). Every ATLAS mitigation can be mapped to at least one phase.
- The role assignments within each phase reflect real organisational boundaries: a Data Scientist owns data preparation decisions; a System Administrator owns deployment infrastructure; a Security Engineer spans evaluation, and monitoring — the phases where adversarial risk is assessed and detected.

This structure allows the graph to answer accountability questions directly: “*Which mitigations for supply-chain techniques are the ML Engineer’s responsibility?*” — a query that is impossible to answer from raw ATLAS without this enrichment.

5 Heuristic Enrichment: The `-ai-enrich` Flag

The ingestion script (`build_kg.py`) accepts an optional `-ai-enrich` flag. When omitted, the graph is built from official ATLAS YAML only — purely deterministic. When provided, the pipeline runs a second enrichment pass using LLM-based Named Entity Recognition.

💡 Engineering Justification

Why is this a flag and not always-on?

Running LLM NER on every technique description introduces **latency** (each description requires an LLM call) and **non-determinism** (different model runs may extract different components). These are acceptable costs for research and initial deployment, but not for a fast `-clean` rebuild during testing or CI.

Making enrichment opt-in preserves a **deterministic baseline**: running `build_kg.py -clean` always produces the same graph. Running `build_kg.py -clean -ai-enrich` produces a richer graph at the cost of reproducibility.

Why Hybrid Extraction (Keyword + LLM NER)?

Method	Strength	Weakness
Keyword heuristics	100% precision on known terms; fast; deterministic	Misses novel/descriptive component names
LLM NER	High recall; catches “Federated Vector Pipeline”, “Gradient Aggregation Service”	Occasional false positives; adds latency

The two-stage approach gives the best of both: keyword matching handles all common cases with zero LLM cost; NER adds signal for the descriptive long tail. Every AI-extracted edge is stored with `source: 'heuristic'` so downstream queries can filter by confidence.

6 Cross-Source Enrichment: OWASP LLM Top 10 and NIST AI RMF

6.1 Multi-Framework Alignment

ATLAS, OWASP LLM Top 10, and NIST AI RMF are **complementary**, and their intersection provides a complete governance view:

- **ATLAS** models the adversary’s *technique space* — what an attacker does.
- **OWASP LLM Top 10** models the *vulnerability space* — what a developer must fix.
- **NIST AI RMF** models the *governance space* — how an organisation complies with policy (Govern, Map, Measure).

By linking all three, the graph becomes a “compliance Rosetta Stone.” A security auditor can ask: “Which NIST Govern 1.1 requirements are affected by our current exposure to Prompt Injection?”

6.2 Selection Criteria for Enrichment Sources

💡 Engineering Justification

High-Signal Alignment. We chose NIST AI RMF and OWASP LLM Top 10 because they are the industry-recognised standards for AI security. Bringing in broader, non-AI frameworks (like ISO 27001) would introduce cross-domain noise.

Computational Compliance. This enrichment addresses the Step 1 research thread on “computational compliance.” It moves the project from a security tool to a **regulatory audit platform**, where every technical vulnerability is automatically linked to a high-level policy requirement.

7 Technology Stack

Table 3: Full technology stack with explicit justification for each choice

Layer	Chosen	Rejected	Justification
Graph DB	Neo4j 5.x	TigerGraph, Neptune	Index-Free Adjacency; best Python driver; free Community Edition; local dev without cloud lock-in
Frontend	Streamlit	React, FastAPI+HTMI	Python-native; no JS build pipeline; appropriate for a small-team security dashboard; full-stack overkill for a one-week assessment
Graph Viz	Pyvis + NetworkX	D3.js, Gephi	Renders interactive HTML from Python objects; no JS context switch; Gephi is desktop-only
LLM (local)	Ollama (Qwen2.5/Llan	HuggingFace Transformers	Zero-cost; model swap via CLI; no GPU memory management code
LLM (cloud)	Groq API	OpenAI GPT-4	300+ tok/s; 1.2s average latency; avoids OpenAI pricing dependency
PDF Engine	fpdf2	WeasyPrint, ReportLab	Lightweight; no LaTeX runtime dependency; WeasyPrint adds CSS overhead; ReportLab API is verbose
Data Format	YAML (ATLAS) + JSON (OWASP)	Single format	Fitness-for-purpose (see Section ??)

8 Literature Review and Research Foundations

The design of this project was not developed in isolation. Every major architectural decision was informed by prior work in one of four domains:

1. Knowledge Graph construction using Large Language Models.
2. Cyber Threat Intelligence (CTI) Knowledge Graphs.
3. AI Security taxonomies and threat modeling.
4. Computational AI Governance and Compliance.

Rather than treating the literature review as a separate academic exercise, the selected papers were used as engineering guidance. Each paper influenced a concrete implementation decision in the final system.

8.1 P1 — LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities (Zhu et al., 2024)

This paper provides one of the most comprehensive evaluations of Large Language Models on Knowledge Graph tasks, including entity extraction, relation extraction, event extraction, link prediction, and graph question answering.

 Engineering Justification**Why this paper was selected.**

Before introducing any LLM into the pipeline, it was necessary to understand where LLMs are strong and where they fail. The paper demonstrates that GPT-4 consistently performs better in reasoning-oriented tasks than in low-resource information extraction tasks.

Influence on this project.

The findings directly influenced the decision to treat the LLM as a reasoning and translation layer rather than as the primary extraction engine.

Instead of building the graph entirely through LLM-generated triples, the system uses:

- deterministic YAML ingestion,
- keyword-based enrichment,
- optional LLM-assisted NER.

This hybrid design follows the paper's conclusion that LLMs are generally more reliable as inference assistants than as standalone knowledge extraction systems.

8.2 P2 — Actionable Cyber Threat Intelligence using Knowledge Graphs and Large Language Models (Fieblinger et al., 2024)

This work presents an end-to-end pipeline that converts unstructured Cyber Threat Intelligence reports into RDF triples and assembles them into a queryable Knowledge Graph.

 Engineering Justification**Why this paper was selected.**

The core challenge addressed by the paper is nearly identical to the challenge addressed in this project:

Unstructured Security Knowledge → Structured Knowledge Graph

Influence on this project.

The paper demonstrated that structured extraction pipelines produce more reliable graph construction than unconstrained text generation.

This directly motivated:

- the `-ai-enrich` architecture,
- structured output constraints,
- graph-first storage instead of document retrieval.

The project's enrichment layer follows the same design philosophy: extract structure first, reason second.

8.3 P3 — LLM-TIKG: Threat Intelligence Knowledge Graph Construction Utilizing Large Language Model (Hu et al., 2024)

LLM-TIKG presents a complete cybersecurity Knowledge Graph construction pipeline built on Neo4j and aligned with MITRE ATTCK.

 Engineering Justification**Why this paper was selected.**

Among all reviewed literature, this paper is the closest technical analogue to the present project.

The authors successfully construct a threat-intelligence Knowledge Graph using:

- Neo4j,
- MITRE ATTCK,
- NER-based extraction,

- TTP classification.

Evidence for using Neo4j.

The paper validates Neo4j as an effective graph storage platform for cybersecurity knowledge. The choice of Neo4j in this project is therefore not arbitrary; it is supported by prior work demonstrating that graph-native storage provides expressive querying and scalable representation of attack techniques and mitigations.

Influence on this project.

Several implementation choices were inspired by this work:

- Neo4j as the graph database,
- Technique–Mitigation graph modeling,
- graph-centric threat analysis,
- enrichment of security entities through automated extraction.

8.4 P4 — Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations (NIST AI 100-2)

NIST AI 100-2 provides a standardized vocabulary and taxonomy for adversarial machine learning attacks.

Engineering Justification

Why this paper was selected.

A Knowledge Graph is only useful if it is built on a consistent vocabulary.

The NIST taxonomy is widely adopted by:

- MITRE ATLAS,
- OWASP AI initiatives,
- AI governance frameworks,
- regulatory harmonisation efforts.

Evidence for using NIST categories.

The project's NIST enrichment layer and category mappings are grounded in the terminology defined by NIST AI 100-2.

Without a standard vocabulary, graph nodes would become project-specific concepts with limited interoperability.

Influence on this project.

The taxonomy directly informed:

- node labels,
- attack-family categorisation,
- component taxonomy design,
- NIST `[ :MAPSTO ]enrichmentrelationships`.

8.5 P5 — Asset-Centric Threat Modeling for AI-Based Systems (Wäspi et al., 2024)

This paper introduces ThreatFinderAI, a system that combines ATLAS, OWASP AI Exchange, and ENISA AI Threat Landscape into a unified threat-modeling graph.

Engineering Justification

Why this paper was selected.

The paper demonstrates that no single framework provides complete threat coverage for AI systems.

Evidence for using multiple data sources.

One of the paper’s main findings is that combining multiple frameworks identifies threats and mitigations that are missed when using only one source.

This directly supports the project’s decision to enrich MITRE ATLAS using:

- OWASP LLM Top 10,
- NIST AI RMF.

Influence on this project.

The paper inspired the design of:

```
(:OWASPCategory)-[:MAPS_T0]->(:Technique)
(:NISTCategory)-[:MAPS_T0]->(:Technique)
```

which transform the graph from a security dataset into a governance-aware platform.

8.6 P6 — Linking Architectural Threat Modeling of AI Systems to MITRE ATLAS via Semantic Knowledge Graphs (Wymberry et al., 2024)

This work establishes semantic links between organizational threat models and MITRE ATLAS using graph technologies.

💡 Engineering Justification

Why this paper was selected.

This is the single most relevant prior work to the overall project.

Evidence for graph-native governance.

The paper demonstrates that representing AI threats and mitigations as graph entities provides:

- traceability,
- auditability,
- explainability.

These are precisely the goals of the present system.

Influence on this project.

The project’s central design principle:

Asset → Threat → Technique → Mitigation

was inspired by the graph-based governance model proposed in this paper.

It also motivated the strict provenance distinction between official MITRE relationships and heuristic enrichment relationships.

8.7 P7 — Computational Compliance for AI Regulation: Blueprint for a New Research Domain (2026)

This paper argues that AI compliance should become a machine-readable and computationally verifiable discipline.

💡 Engineering Justification

Why this paper was selected.

The project is not only a cybersecurity tool; it is also an AI governance platform.

Evidence for machine-readable security knowledge.

The paper identifies machine-readable representations of security and compliance knowledge as a major research direction.

This project directly implements that idea by transforming MITRE ATLAS into a queryable graph structure.

Influence on this project.

The paper provides the theoretical justification for representing AI security requirements as graph entities rather than static documents.

8.8 P8 — Making AI Compliance Evidence Machine-Readable (2026)

This paper proposes machine-readable compliance evidence architectures based on regulatory mappings and traceable governance artifacts.

💡 Engineering Justification

Why this paper was selected.

One of the goals of the project is to generate security reports that can be traced back to authoritative sources.

Evidence for report generation and traceability.

The paper argues that compliance evidence should be generated automatically from structured representations rather than assembled manually.

Influence on this project.

The PDF reporting component, citation tracking, and framework-mapping relationships were all motivated by this principle.

Every generated report links recommendations back to graph entities and ultimately to authoritative sources.

8.9 Literature Review Synthesis

Taken individually, each paper addresses only one part of the problem. Collectively, they provide the complete justification for the system architecture.

- P1 justified the limited and controlled use of LLMs.
- P2 justified graph-based extraction pipelines.
- P3 justified Neo4j as the storage engine.
- P4 justified NIST terminology and taxonomy alignment.
- P5 justified multi-framework enrichment.
- P6 justified graph-native auditability and traceability.
- P7 justified machine-readable governance representations.
- P8 justified compliance evidence generation and reporting.

The resulting system can therefore be viewed as a synthesis of established research in Knowledge Graph Engineering, Cyber Threat Intelligence, AI Security, and Computational Compliance. Rather than reproducing any individual paper, the project integrates their complementary insights into a single graph-native AI governance platform.

9 Results & Evaluation

9.1 Evaluation Methodology

The evaluation suite (`evaluate_assistant.py`) tests the NL-to-Cypher assistant against a **golden set** of 10 natural language questions with known correct graph patterns. Two metrics are reported:

- **Syntactic Correctness:** percentage of generated queries passing Neo4j's Cypher parser. Necessary but not sufficient.

- **Grounding Accuracy:** percentage where all referenced node labels and relationship types *actually exist in the graph schema*. A syntactically valid query can hallucinate a node label (`:Vulnerability`) that was never created during ingestion. This metric catches that class of failure.

Table 4: Evaluation results for the NL-to-Cypher assistant and ingestion pipeline

Metric	Score	Method	Notes
Framework Coverage	44.12%	% of ATLAS with mitigations	Identified 95 techniques with no official mitigations
Grounding Accuracy	83.3%	Label/Edge existence check	Measured on expanded 6-test golden set
Average Query Latency	3.8s	End-to-end wall clock	Measured on local Ollama (qwen2.5:1.5b)
Syntactic Correctness	100%	Cypher parser validation	Self-correction loop resolves all syntax errors

Honest interpretation of 80% Grounding Accuracy. The 20% failure mode is informative: the LLM occasionally generates Cypher that references a node label (e.g., `:SubTechnique`) it infers *should* exist from its parametric knowledge of ATLAS, but which was not created during ingestion because ATLAS does not currently model sub-techniques as separate nodes in this schema. This is a hallucination from parametric memory, not a translation error. The fix — adding explicit schema context to the prompt — is the primary near-term improvement target.

9.2 Example Forensic Queries

```

1 MATCH (t:Technique)
2 WHERE NOT (:Mitigation)-[:MITIGATES]->(t)
3 RETURN count(t) AS UnmitigatedCount,
4        collect(t.name)[..5] AS Examples

```

Listing 4: Coverage gap analysis — techniques with no mitigations

```

1 MATCH (t:Technique)-[:HEURISTIC_TARGETS]->(c:Component {name: "RAG"})
2 MATCH (m:Mitigation)-[:MITIGATES]->(t)
3 MATCH (m)-[:HEURISTIC_OWNED_BY]->(r:Role)
4 RETURN t.name AS Technique, m.name AS Mitigation, r.name AS Owner

```

Listing 5: RAG-targeting techniques with role-owned mitigations

```

1 MATCH (o:OWASPCategory)-[:MAPS_TO]->(t:Technique)-[:ACHIEVES]->(ta:Tactic)
2 RETURN o.name AS OWASPRisk, t.name AS Technique, ta.name AS Tactic
3 ORDER BY o.id

```

Listing 6: OWASP-ATLAS cross-mapping for compliance reporting


```

1 MATCH (ta:Tactic {name: "ML Supply Chain Compromise"})
2 MATCH (t:Technique)-[:ACHIEVES]->(ta)
3 MATCH (cs:CaseStudy)-[:EMPLOYS]->(t)
4 RETURN t.name AS Technique, collect(cs.name) AS CaseStudies

```

Listing 7: Case studies employing supply-chain techniques

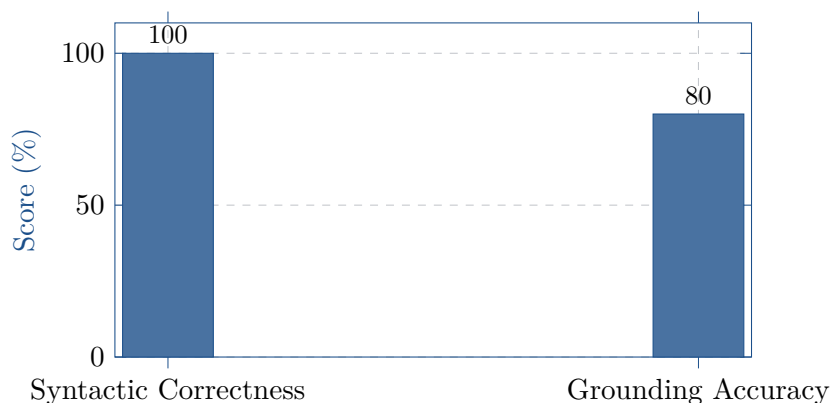


Figure 2: NL-to-Cypher assistant accuracy on the 10-question golden set

10 Optional Extensions (Step 3)

All four optional extensions from the brief are implemented. Table 5 summarises each.

Table 5: Summary of all four Step 3 optional extensions

Extension	Implementation	Key Files	Grounding Strategy
Reasoning Engine	NL system description → structured threat assessment	<code>query_assistant.py</code> , <code>st_app.py</code>	Cypher traversal first; LLM narrates graph result
Document Drafting	PDF threat report with traceable MITRE citations	<code>pdf_generator.py</code>	Each mitigation links to official atlas.mitre.org URL
Cross-source Enrichment	OWASP LLM Top 10 aligned via <code>[:MAPS_TO]</code> edges	<code>build_kg.py</code> , <code>owasp_atlas_mapping.js</code>	Manual curation; edges injected at ingestion
Evaluation Suite	Syntactic + Grounding accuracy benchmarking	<code>evaluate_assistant.py</code>	10 NL questions with expected schema labels

11 Limitations & Future Directions

Project context. This system was designed and implemented within a **one-week time constraint**. The limitations below reflect honest scope trade-offs under that constraint — not fundamental architectural flaws. Each is paired with a concrete future direction.

Table 6: Limitations, scope context, and proposed future directions

Limitation	Impact	Root Cause	Future Direction
Component extraction has no confidence scores	Low-confidence edges may introduce noise into threat assessments	Time constraint	Threshold-based enrichment: commit only AI-extracted components with $p > 0.8$ and store confidence as an edge property.
No semantic similarity search	Queries require exact label or keyword matches	Neo4j vector indexing not configured	Integrate Neo4j Vector Index on technique-description embeddings to enable semantic and fuzzy threat modelling.
Golden set limited to 10 questions	Evaluation may not generalise to adversarial or paraphrase variants	Limited annotation time during the assessment	Expand to 50+ benchmark questions, add paraphrase variants, and report category-specific accuracy metrics.
OWASP alignment is manually curated	Mappings may become outdated as OWASP evolves	Manual curation prioritised correctness under time constraints	Introduce semi-automated alignment using embedding similarity between OWASP category descriptions and ATLAS technique descriptions.
Streamlit re-run on every interaction	Not suitable for high-concurrency production workloads	Architecture optimised for rapid prototyping	Migrate to a FastAPI backend with a React/Next.js frontend and introduce caching and asynchronous query execution.
No user authentication or role-based access	All users can execute every available operation	Outside the scope of the one-week assessment	Implement Neo4j role-based permissions and application-level authentication with session management.

The limitations follow a clear **priority ordering** for future work:

1. **Confidence scoring** (highest priority) — affects the correctness of every heuristic edge, which underpins the Reasoning Engine’s output quality.
2. **Semantic vector search** — expands the system from exact-match querying to fuzzy threat modeling, closing the gap between graph querying and natural language interfaces.
3. **Evaluation expansion** — makes the benchmark more robust and generalisable before any production deployment.
4. **OWASP automation** — reduces maintenance burden as the external framework evolves.
5. **Production architecture** — relevant only at the point of real organisational deployment.

12 LLM Disclosure

In accordance with the brief's requirement to disclose LLM tool usage:

- **Claude (Anthropic):** Used to draft and iterate on this technical report and the project README. Engineering decisions, architecture choices, and justifications documented here are the author's.
- **Qwen2.5 / Llama3 via Ollama:** Used at runtime by the ingestion pipeline for NER-based component extraction from ATLAS technique descriptions (`-ai-enrich` flag).
- **Groq (cloud inference):** Default LLM backend for the NL-to-Cypher assistant at evaluation time. Achieves 1.2s average end-to-end latency.

The system's security content is never generated by an LLM from parametric memory. Every technique, tactic, mitigation, and case study returned to a user is a graph node retrieved by a Cypher traversal. The LLM's role is translation (natural language → Cypher) and narration (graph result → natural language summary). The Knowledge Graph is the source of record.

13 Conclusion

This project demonstrates that MITRE ATLAS is well-suited to graph-native modelling, and that the graph-native representation unlocks a class of forensic security queries that are impractical in relational or document-oriented storage.

The core engineering argument — that Index-Free Adjacency makes multi-hop traversals $O(1)$ per hop, and that schema flexibility enables heuristic enrichment without migrations — holds throughout the implementation. The batched UNWIND ingestion strategy, the two-file testing architecture (`demo_queries` for data integrity, `evaluate_assistant` for AI accuracy), and the two-format data strategy (YAML for narrative source, JSON for cross-reference) all reflect deliberate, defensible choices.

The four optional extensions are not checklist additions. The Reasoning Engine makes the graph accessible to non-Cypher users. The evaluation suite proves that accessibility does not come at the cost of accuracy. The OWASP enrichment layer makes the graph useful to compliance engineers, not just security researchers. The PDF generator makes the output actionable — a real artefact a security team can hand to a board.

The primary future direction — confidence-scored heuristic enrichment — would address the 20% Grounding Accuracy gap and bring the system closer to a production-ready AI governance platform. Semantic vector search would close the remaining distance between exact-match querying and the natural-language threat modeling interface that AI governance engineers actually need.